1. **What is Java?**
   Java is a high-level, object-oriented programming language developed by Sun Microsystems (now Oracle). It follows the principle of "Write Once, Run Anywhere" (WORA) due to its platform independence.

2. **Why is Java called platform independent?**
   Java source code is compiled into bytecode, which runs on the Java Virtual Machine (JVM). The JVM interprets bytecode to the native machine code of the underlying platform, allowing the same bytecode to run on any operating system that has a JVM.

3. **What is the difference between JDK, JRE, and JVM?**

- **JVM (Java Virtual Machine)**: Executes bytecode and provides runtime environment.
- **JRE (Java Runtime Environment)**: Includes JVM + core libraries + other components to run Java applications.
- **JDK (Java Development Kit)**: Includes JRE + development tools (compiler, debugger, etc.) to develop Java applications.

4. **What is bytecode in Java?**
   Bytecode is the intermediate code generated by the Java compiler (javac) from .java source files. It is platform-independent and executed by the JVM.

5. **What are the main features of Java?**
   Object-oriented, platform independent, secure, robust, multithreaded, portable, high performance, dynamic, and interpreted (via JIT compilation).

6. **What is object-oriented programming?**
   OOP is a programming paradigm that organizes software design around objects, which contain data (fields) and behavior (methods). Key principles: encapsulation, inheritance, polymorphism, abstraction.

7. **Why is Java considered secure?**

- Bytecode verification ensures no illegal code runs.
- No explicit pointer manipulation.
- Security manager and classloaders control resource access.
- Runtime checks (array bounds, type casting) prevent many vulnerabilities.

8. **What is the role of the JVM?**
   The JVM loads, verifies, and executes bytecode. It provides memory management (heap, stack, garbage collection), runtime environment, and translates bytecode into native machine code using JIT compilation.

9. **What is the difference between compiled and interpreted languages?**

- **Compiled**: Entire source code is translated to machine code before execution (e.g., C, C++). Faster execution, platform-specific.
- **Interpreted**: Code is executed line-by-line by an interpreter at runtime (e.g., Python, JavaScript). Easier debugging, slower.

10. **What is the** main() **method in Java?**
    The entry point of any standalone Java application. Its signature: public static void main(String[] args). public for JVM access, static to call without object, void returns nothing.

11. **What are variables in Java?**
    Named memory locations that store data values. Each variable has a type, name, and value. Types include primitive (int, double, etc.) and reference (objects, arrays).

12. **What are primitive data types in Java?**
    byte, short, int, long, float, double, char, boolean. They hold raw values directly.

13. **What is the difference between** int **and** float**?**
    int stores whole numbers (32-bit signed), no decimal. float stores floating-point numbers (32-bit IEEE 754), allows decimals. Memory representation and precision differ.

14. **What is type casting?**
    Converting one data type to another. Two types: **implicit** (widening, automatic) and **explicit** (narrowing, manual using cast operator).

15. **What is implicit type conversion?**
    Automatic conversion of a smaller data type to a larger compatible type (e.g., int to long). No data loss, done by compiler.

16. **What is explicit type conversion?**
    Manual conversion using cast operator (targetType). Used for narrowing (e.g., double to int), may cause data loss. int x = (int) 3.14;

17. **What is the difference between local and global variables?**

- **Local**: Declared inside method/block, scope limited to that block, no default value, must be initialized before use.
- **Global (instance/class variables)**: Declared inside class but outside methods, have default values, accessible throughout class.

18. **What are constants in Java?**
    Variables whose value cannot change after assignment. Declared using final keyword (e.g., final double PI = 3.1416;). Naming convention: uppercase with underscores.

19. **What is the** final **keyword?**

- For variable: makes it a constant.
- For method: prevents overriding.
- For class: prevents inheritance (cannot be subclassed).

20. **What are wrapper classes?**
    Classes that wrap primitive types into objects.
    Examples: Integer, Double, Character, Boolean. Used in collections, generics, and when object is required.

21. **What are arithmetic operators?**
    +, -, *, /, % (modulus). Used for mathematical operations on numeric types.

22. **What are relational operators?**
    ==, !=, >, <, >=, <=. Compare two values and return a boolean result.

23. **What are logical operators?**
    && (AND), || (OR), ! (NOT). Operate on boolean expressions. Support short-circuit evaluation for && and ||.

24. **What is the difference between** && **and** ||**?**
    && returns true only if both operands are true. || returns true if at least one operand is true. Both short-circuit: && stops at first false, || stops at first true.

25. **What are assignment operators?**
    =, +=, -=, *=, /=, %=, etc. Assign a value to a variable. Compound assignments perform operation and assignment in one step.

26. **What are unary operators?**
    Operate on a single operand: + (unary plus), - (unary minus), ++ (increment), --- (decrement), ! (logical NOT).

27. **What is the ternary operator?**
    Shorthand for if-else. Syntax: condition ? expression1 : expression2.
    Returns expression1 if condition true, else expression2.

28. **What is operator precedence?**
    The order in which operators are evaluated. Example: multiplication before addition. Parentheses () have highest precedence. Use precedence table or parentheses for clarity.

29. **What is short-circuit evaluation?**
    In logical && and ||, the right operand is not evaluated if the left operand determines the result. For &&, if left is false, result is false (right skipped). For ||, if left is true, result is true (right skipped).

30. **What is the** instanceof **operator?**
    Checks whether an object is an instance of a specific class, subclass, or interface. Returns boolean. Example: if (obj instanceof String)

31. **What is an** if **statement?**
    A conditional control statement that executes a block of code only if a boolean expression evaluates to true. Can be extended with else if and else.

32. **Difference between** if-else **and** switch**.**

- if-else: Can test any boolean condition (ranges, complex expressions).
- switch: Tests equality of a single variable against multiple constant values. Faster for many discrete values (int, char, String, enum).

33. **What is a nested** if **statement?**
    An if statement inside another if or else block. Used for checking multiple conditions sequentially.

34. **What is a loop?**
    A control structure that repeats a block of code multiple times based on a condition. Types: for, while, do-while, enhanced for.

35. **Difference between** for**,** while**, and** do-while**.**

- for: When number of iterations known (initialization, condition, increment together).
- while: When iterations unknown, condition checked before loop body.
- do-while: Body executes at least once, condition checked after.

36. **What is an infinite loop?**
    A loop whose termination condition never becomes false.
    Example: while(true){} or for(;;){}. Usually unintended, but can be used with break for controlled exit.

37. **What does the** break **statement do?**
    Exits the current loop or switch statement immediately. Control flows to the next statement after the loop/switch.

38. **What does the** continue **statement do?**
    Skips the current iteration of a loop and moves to the next iteration (evaluates condition again). Does not exit the loop.

39. **What is the enhanced for loop?**
    Also called "for-each" loop. Syntax: for (type var : array/collection). Simplifies iteration over arrays or Iterable objects without manual index management.

40. **When should you use a** switch **statement?**
    When testing a single variable (byte, short, int, char, String, enum) against several constant values. Cleaner and often more efficient than long if-else-if chains.

41. **What is an array?**
    A fixed-size, indexed, homogeneous collection of elements stored in contiguous memory locations. All elements share the same data type.

42. **How do you declare an array in Java?**
    int[] numbers; or int numbers[];. Then initialize: numbers = new int[5]; or combine: int[] numbers = new int[5];. Use array initializer: int[] nums = {1,2,3};

43. **What is the difference between 1D and 2D arrays?**
    1D array stores elements in a single row/line. 2D array is an array of arrays (like a matrix with rows and columns). Accessed by two indices: array[row][col].

44. **What are jagged arrays?**
    A 2D array where each row can have a different number of columns. Example: int[][] jagged = new int[3][]; then assign variable-length second dimensions.

45. **How are arrays stored in memory?**
    Arrays are objects stored in heap memory. The array variable holds a reference to the heap location. Elements are stored contiguously (for primitive arrays, values directly; for object arrays, references to objects).

46. **What is array indexing?**
    Accessing individual elements using an integer index. In Java, indices start at 0 (first element) and go up to length-1 (last element).

47. **What happens if array goes out of bounds?**
    JVM throws ArrayIndexOutOfBoundsException at runtime. Program terminates unless exception is handled.

48. **How do you find the length of an array?**
    Using the length attribute (not method): array.length. For 2D arrays, array.length gives number of rows.

49. **What are the limitations of arrays?**

- Fixed size after creation.
- Cannot store mixed types (homogeneous).
- No built-in methods for search, sort, etc.
- Only length property.

50. **Difference between arrays and** ArrayList**.**

- **Array**: Fixed size, can hold primitives and objects, direct access with [], less memory overhead.
- **ArrayList**: Resizable, only objects (use wrapper for primitives), provides methods like add(), remove(), size(), part of Collections Framework.

51. **What is a** String **in Java?**
    A sequence of characters (immutable) represented as objects of java.lang.String class. String literals are stored in the String pool for efficiency.

52. **Difference between** String **and** StringBuilder**.**

- String: Immutable – any change creates a new object.
- StringBuilder: Mutable – modifications change the same object. More efficient for repeated concatenation.

53. **What is immutability in Java Strings?**
    Once a String object is created, its value cannot be changed. Any operation that appears to modify the string (e.g., concat(), replace()) returns a new String object.

54. **What is the String pool?**
    A special memory area in the heap that stores unique string literals. When a string literal is created, JVM checks the pool; if exists, reference is reused, saving memory.

55. **Difference between** == **and** .equals()**.**

- ==: Compares references (memory addresses) for objects. For primitives, compares values.
- .equals(): Compares content (actual characters) for strings. Override in custom classes for logical equality.

56. **What are common** String **methods?**
    length(), charAt(), substring(), indexOf(), toLowerCase(), toUpperCase(), trim(), replace(), split(), equals(), compareTo(), startsWith(), endsWith().

57. **What does** trim() **do?**
    Removes leading and trailing whitespace (space, tab, newline) from a string. Does not remove spaces in the middle. Returns a new string.

58. **Difference between** substring() **and** split()**.**

- substring(int beginIndex, int endIndex): Extracts a portion of the string between indices.
- split(String regex): Breaks the string into an array of substrings based on a delimiter regex.

59. **What is string concatenation?**
    Combining two or more strings into one. Use + operator or concat() method. For many concatenations, prefer StringBuilder for performance.

60. **Why are Strings immutable?**

- Security (e.g., database usernames, passwords).
- Thread safety – no synchronization needed.
- String pool works only because of immutability.
- Performance – caching hashcode for HashMap keys.

61. **What is a class?**
    A blueprint or template that defines data (fields) and behavior (methods) for objects. It encapsulates state and functionality.

62. **What is an object?**
    An instance of a class, created using new keyword. It has its own copy of instance variables and can invoke methods of its class.

63. **What is a constructor?**
    A special method with the same name as the class, invoked when an object is created. It initializes the object's state. No return type, can be overloaded.

64. **Difference between constructor and method.**

- **Constructor**: Called automatically at object creation, same name as class, no return type, used for initialization.
- **Method**: Explicitly called, any name, has return type (or void), performs actions.

65. **What is method overloading?**
    Multiple methods in the same class with the same name but different parameter lists (type, number, or order). Compile-time polymorphism. Return type can differ but not sufficient.

66. **What is method overriding?**
    Subclass provides a specific implementation of a method already defined in its superclass. Method signature must be same (name + parameters). Runtime polymorphism. Use @Override annotation.

67. **What is inheritance?**
    Mechanism where one class (subclass/child) acquires fields and methods of another class (superclass/parent). Promotes code reuse and establishes "is-a" relationship. Use extends keyword.

68. **Types of inheritance in Java.**

- Single (one subclass, one superclass) – only type supported for classes.
- Multilevel (A→B→C).
- Hierarchical (one parent multiple children).
- Multiple inheritance for classes not allowed, but supported for interfaces using implements.

69. **What is polymorphism?**
    Ability of an object to take many forms. Two types:

- **Compile-time (static)**: Method overloading.
- **Runtime (dynamic)**: Method overriding (decided at runtime based on actual object type).

70. **What is abstraction?**
    Hiding implementation details and exposing only essential features. Achieved using abstract classes and interfaces. Use abstract keyword for methods/classes without body.

71. **What is encapsulation?**
    Bundling data (variables) and methods that operate on that data into a single unit (class), and restricting direct access. Achieved using private fields and public getters/setters.

72. **What are access modifiers?**
    Keywords that set the visibility/accessibility of classes, methods, and fields. Four levels: private, default (package-private), protected, public.

73. **Difference between** public**,** private**, and** protected**.**

- public: Accessible everywhere.
- private: Only within the same class.
- protected: Within same package + subclasses (even outside package).
- default (no modifier): Only within same package.

74. **What is the** this **keyword?**
    Reference to the current object. Used to:

- Access instance variables when shadowed by parameters.
- Call another constructor of same class (this(...)).
- Pass current object as argument.

75. **What is the** super **keyword?**
    Reference to the immediate parent class object. Used to:

- Access parent class fields/methods (especially overridden ones).
- Invoke parent class constructor (super(...)) – must be first statement in child constructor.

76. **What is an abstract class?**
    A class declared with abstract keyword. Can have abstract (no body) and concrete methods. Cannot be instantiated. Must be extended; subclasses must implement abstract methods unless also abstract.

77. **What is an interface?**
    A contract that declares abstract methods (and since Java 8, default/static methods). A class implements an interface and must provide method bodies. Supports multiple inheritance. Cannot have instance variables (only public static final constants).

78. **Difference between abstract class and interface.**

- **Abstract class**: Can have state (instance variables), constructors, concrete methods, single inheritance.
- **Interface**: Only public abstract methods (before Java 8), no constructors, no instance variables, supports multiple inheritance. Since Java 8: default/static methods; Java 9: private methods.

79. **Can Java support multiple inheritance?**
    For classes – no (to avoid diamond problem). For interfaces – yes, a class can implement multiple interfaces. Since Java 8, default methods in interfaces can cause conflicts, resolved by overriding.

80. **What is dynamic method dispatch?**
    Mechanism by which a call to an overridden method is resolved at runtime based on the actual object type (not reference type). Example: Parent p = new Child(); p.display(); – Child's method runs.

81. **What is an exception?**
    An event that disrupts normal program flow. Represented by objects of Throwable subclasses (Exception, Error). Can be handled using try-catch or declared with throws.

82. **Difference between checked and unchecked exceptions.**

- **Checked**: Compiler forces handling (try-catch or throws). Subclasses of Exception except RuntimeException. Example: IOException, SQLException.

- **Unchecked**: Not checked at compile time. Subclasses of RuntimeException. Example: NullPointerException, ArithmeticException. Optionally handled.

83. **What is** try-catch**?**
    try block encloses code that may throw an exception. catch block handles the specific exception type. Multiple catch blocks allowed. finally optional.

84. **What is the** finally **block?**
    A block that always executes after try/catch, regardless of whether an exception occurred or not. Typically used for cleanup (closing files, releasing resources).

85. **What is the** throw **keyword?**
    Used to explicitly throw an exception from a method or block. Example: throw new ArithmeticException("Division by zero");

86. **What is the** throws **keyword?**
    Used in method declaration to indicate that the method might throw one or more checked exceptions. The caller must handle or declare them further.

87. **What is exception propagation?**
    If an exception is not caught in the current method, it propagates up the call stack until caught or reaches main() – then program terminates. Unchecked exceptions propagate automatically.

88. **What is a custom exception?**
    User-defined exception class extending Exception (checked)
    or RuntimeException (unchecked). Used to represent application-specific error conditions.

89. **Why is exception handling important?**
    Prevents abrupt termination, separates error-handling code from normal logic, allows graceful recovery, and provides meaningful error messages to users or logs.

90. **What happens if an exception is not handled?**

- Checked exception: Compilation error.
- Unchecked exception: Runtime termination with stack trace printed to standard error.

91. **What is multithreading?**
    Concurrent execution of two or more threads (lightweight subprocesses) within a single program. Java provides Thread class and Runnable interface for creating threads.

92. **Difference between process and thread.**

- **Process**: Independent program in its own memory space, heavyweight, separate resources.
- **Thread**: Lightweight unit of execution within a process, shares memory and resources, easier communication.

93. **What is synchronization?**
    Mechanism to control access to shared resources by multiple threads. Prevents race conditions. Achieved using synchronized keyword (methods or blocks) or Lock interfaces.

94. **What is garbage collection?**
    Automatic memory management that deallocates objects no longer referenced. JVM runs garbage collector periodically. System.gc() requests but does not guarantee execution.

95. **What is the purpose of the** static **keyword?**
    Denotes class-level members (variables, methods, blocks) that belong to the class rather than instances. Static members accessed using class name. Shared across all objects.

96. **Difference between static and non-static members.**

- **Static**: One copy per class, loaded at class loading, accessed via class name, cannot access instance members directly.
- **Non-static (instance)**: Each object has own copy, accessed via object reference, can access static members.

97. **What is recursion?**
    A method that calls itself. Each call must have a base case to terminate. Useful for problems like factorial, Fibonacci, tree traversals. Risk of stack overflow if too deep.

98. **What are packages in Java?**
    Namespaces that group related classes and interfaces. Avoid naming conflicts, control access, and organize code. import statement used to access classes from other packages.

99. **What is the difference between** import **and** package**?**

- package: Declares the package for the current file (must be first non-comment line).
- import: Allows using classes from other packages without fully qualified names.

100. **Why is Java popular in software development?**

- Platform independence (JVM).
- Rich standard library (APIs).

- Strong memory management (garbage collection).
- Multithreading support.
- Large ecosystem (Spring, Android, Hadoop).
- Backward compatibility.
- Strong community and corporate backing (Oracle).